



UNIVERSIDADE KIMPA VITA

INSTITUTO POLITÉCNICO

CURSO DE LICENCIATURA EM ENGENHARIA INFORMÁTICA

RELATÓRIO DO TRABALHO EM GRUPO

DE SGBD II

TEMA: Implementação de Mecanismos de Transações e Propriedades ACID
em um Sistema de Vendas

CLASSE: 3º ANO

Integrantes do Grupo:

1. **António Almeida K. Nzinga**
2. **Kiassisua Simão Pedro**
3. **Luís Mucuzo António**
4. **Maria Tumba Domingos João**
5. **Suzana Diasivi Nicolau**

DOCENTE

Moyo Kanivengidio

1. Introdução	1
2. Fundamentação Teórica: O Modelo ACID	1
3. Implementação Prática: DDL e DML da Base de Dados	1
4. O Núcleo do Relatório: Logica de Transação Controlada.....	5
5. Simulação e Análise de Cenários Práticos	8
5.1 Cenário 1: Carga Inicial de Dados.....	8
5.2 Cenário 2: Execução com Sucesso (Fluxo do COMMIT).....	9
5.3 Cenário 3: Aborto por Falha de Regra de Negócio (Fluxo do ROLLBACK).....	9
6. Conclusão	10

1. Introdução

No contexto dos Sistemas de Gerenciamento de Banco de Dados Relacionais (SGBDR), a execução segura e fiável de operações concorrentes é um dos pilares fundamentais para garantir a integridade dos dados organizacionais. Este relatório apresenta o desenvolvimento e a implementação do banco de dados “SistemaVendasMercado”, com foco principal no estudo e aplicação prática de Transações Controladas.

O cenário escolhido simula o fluxo comercial de um mercado, englobando o controlo de acessos (Roles e Funcionários), o cadastro de Clientes e Produtos, e o fluxo crítico de registo de Vendas. O objetivo principal é demonstrar como o SGBD impede a perda de consistência dos dados em cenários de falhas operacionais ou falta de stock, utilizando blocos de transação explícita (BEGIN TRANSACTION, COMMIT e ROLLBACK).

2. Fundamentação Teórica: O Modelo ACID

Para garantir que as operações críticas sobre o banco de dados sejam executadas de forma segura, o SGBD baseia-se no acrónimo ACID:

- **Atomicidade:** Garante que todas as instruções contidas numa transação sejam executadas com sucesso. Se qualquer instrução falhar, toda a transação é abortada e o banco de dados retorna ao estado original. No nosso sistema, ou a venda é inserida, o detalhe é gerado e o stock é reduzido por completo, ou nada é guardado.
- **Consistência:** Garante que uma transação leve o banco de dados de um estado válido para outro estado válido, respeitando todas as regras de integridade (regras de negócio e chaves estrangeiras). Por exemplo, o stock de um produto nunca pode ficar negativo.
- **Isolamento:** Garante que transações executadas simultaneamente (concorrentes) não interfiram umas nas outras. O SQL Server utiliza mecanismos de *Locks* (bloqueios) para isolar as operações até que sejam concluídas.
- **Durabilidade:** Garante que, uma vez que a transação receba a confirmação de sucesso (COMMIT), os dados tornam-se permanentes no armazenamento físico, não sendo perdidos mesmo em caso de falha de energia subsequente.

3. Implementação Prática: DDL e DML da Base de Dados

Abaixo apresenta-se o script SQL completo utilizado para estruturar o banco de dados, criar os relacionamentos essenciais e carregar os dados iniciais.

SQL

USE master;

GO

-- 1. Criação e Preparação do Banco de Dados

```
IF EXISTS (SELECT name FROM sys.databases WHERE name =  
'SistemaVendasMercado')
```

BEGIN

```
    ALTER DATABASE SistemaVendasMercado SET SINGLE_USER WITH  
    ROLLBACK IMMEDIATE;
```

```
    DROP DATABASE SistemaVendasMercado;
```

END

GO

```
CREATE DATABASE SistemaVendasMercado;
```

GO

```
USE SistemaVendasMercado;
```

GO

-- 2. Tabela de Roles (Níveis de Acesso)

```
CREATE TABLE Roles (
```

```
    RoleID INT IDENTITY(1,1) PRIMARY KEY,
```

```
    NomeRole VARCHAR(20) NOT NULL UNIQUE
```

```
);
```

```
INSERT INTO Roles (NomeRole) VALUES ('Admin'), ('Gestor'), ('Vendedor');
```

-- 3. Tabela de Usuários (Acesso ao Sistema)

```
CREATE TABLE Usuarios (
```

```
    ID INT IDENTITY(1,1) PRIMARY KEY,
```

```
    Login VARCHAR(50) NOT NULL UNIQUE,
```

```
Senha VARCHAR(50) NOT NULL,  
Nome VARCHAR(50) NOT NULL,  
Nivel VARCHAR(50) NOT NULL  
);
```

```
INSERT INTO Usuarios(Login, Senha, Nome, Nivel) VALUES ('simao', '1234',  
'Kiassisua Simão', 'Admin');
```

-- 4. Tabela de Funcionários

```
CREATE TABLE Funcionarios (  
    FuncionarioID INT IDENTITY(1,1) PRIMARY KEY,  
    Nome VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) NOT NULL UNIQUE,  
    Telefone VARCHAR(20) NOT NULL UNIQUE,  
    Username VARCHAR(50) NOT NULL UNIQUE,  
    Senha VARCHAR(100) NOT NULL,  
    RoleID INT NOT NULL FOREIGN KEY REFERENCES Roles(RoleID),  
    DataCadastro DATETIME DEFAULT GETDATE(),  
    Ativo BIT DEFAULT 1  
);  
  
INSERT INTO Funcionarios (Nome, Email, Telefone, Username, Senha, RoleID)  
VALUES ('Administrador', 'kiasisuasimaop@gmail.com', '942989381', 'Simão', '1234',  
1);
```

-- 5. Tabela de Clientes

```
CREATE TABLE Clientes (  
    ClienteID INT IDENTITY(1,1) PRIMARY KEY,  
    Nome VARCHAR(100) NOT NULL,
```

```
Telefone VARCHAR(20) NOT NULL UNIQUE,  
Email VARCHAR(100) UNIQUE,  
Endereco VARCHAR(200)  
);
```

-- 6. Tabela de Produtos

```
CREATE TABLE Produtos (  
    ProdutoID INT IDENTITY(1,1) PRIMARY KEY,  
    NomeProduto VARCHAR(100) NOT NULL,  
    Categoria VARCHAR(50),  
    PrecoUnitario DECIMAL(18,2) NOT NULL,  
    QuantidadeStock INT NOT NULL DEFAULT 0,  
    DataCadastro DATETIME DEFAULT GETDATE()  
);
```

-- 7. Tabela de Vendas (Cabeçalho da Transação)

```
CREATE TABLE Vendas (  
    VendaID INT IDENTITY(1,1) PRIMARY KEY,  
    ClienteID INT FOREIGN KEY REFERENCES Clientes(ClienteID),  
    FuncionarioID INT FOREIGN KEY REFERENCES Funcionarios(FuncionarioID),  
    DataVenda DATETIME DEFAULT GETDATE(),  
    TotalVenda DECIMAL(18,2) NOT NULL  
);
```

-- 8. Tabela de Detalhes da Venda (Registos Filhos)

```
CREATE TABLE DetalhesVenda (  
    DetalheID INT IDENTITY(1,1) PRIMARY KEY,  
    VendaID INT FOREIGN KEY REFERENCES Vendas(VendaID),
```

```

    ProdutoID INT FOREIGN KEY REFERENCES Produtos(ProdutoID),

    Quantidade INT NOT NULL,

    PrecoUnitario DECIMAL(18,2) NOT NULL,

    SubTotal AS (Quantidade * PrecoUnitario)

);

GO

-- 9. Stored Procedure para Autenticação

CREATE PROC sp_Login

    @Username VARCHAR(50),

    @Senha VARCHAR(100)

AS

BEGIN

    SELECT F.FuncionarioID, F.Nome, R.NomeRole, F.RoleID

    FROM Funcionarios F

    INNER JOIN Roles R ON F.RoleID = R.RoleID

    WHERE F.Username = @Username AND F.Senha = @Senha AND F.Ativo = 1

END

GO

```

4. O Núcleo do Relatório: Logica de Transação Controlada

Para realizar vendas de forma robusta e atômica, foi criada a Stored Procedure `sp_RegistarVenda`. Ela utiliza uma estrutura estruturada de tratamento de exceções (`BEGIN TRY ... BEGIN CATCH`) combinada com comandos de transação explícita do SQL Server.

```

SQL

CREATE PROCEDURE sp_RegistarVenda

    @ClienteID INT,

```

```

@FuncionarioID INT,

@ProdutoID INT,

@QuantidadeVendida INT

AS

BEGIN

    SET XACT_ABORT ON; -- Interrompe a execução imediatamente se houver erro grave

    SET NOCOUNT ON;


    DECLARE @PrecoAtual DECIMAL(18,2);

    DECLARE @StockDisponivel INT;

    DECLARE @NovaVendaID INT;


    -- Capturar o preço e stock em tempo real

    SELECT @PrecoAtual = PrecoUnitario, @StockDisponivel = QuantidadeStock

    FROM Produtos

    WHERE ProdutoID = @ProdutoID;


    -- Validação de Regra de Negócio: Impedir stock negativo (Consistência)

    IF @StockDisponivel < @QuantidadeVendida

    BEGIN

        RAISERROR('Erro: Stock insuficiente para realizar esta venda.', 16, 1);

        RETURN;

    END


    -- INÍCIO DA TRANSAÇÃO

```



```

BEGIN TRANSACTION;

BEGIN TRY

    -- Passo A: Inserir registo na tabela Pai (Vendas)

    INSERT INTO Vendas (ClienteID, FuncionarioID, DataVenda, TotalVenda)

    VALUES (@ClienteID, @FuncionarioID, GETDATE(), 0);


    SET @NovaVendaID = SCOPE_IDENTITY(); -- Captura o ID gerado
    automaticamente


    -- Passo B: Inserir registo na tabela Filho (DetalhesVenda)

    INSERT INTO DetalhesVenda (VendaID, ProdutoID, Quantidade, PrecoUnitario)

    VALUES (@NovaVendaID, @ProdutoID, @QuantidadeVendida, @PrecoAtual);


    -- Passo C: Atualizar o valor consolidado da venda

    UPDATE Vendas

    SET TotalVenda = (@QuantidadeVendida * @PrecoAtual)

    WHERE VendaID = @NovaVendaID;


    -- Passo D: Reduzir fisicamente o stock do Mercado

    UPDATE Produtos

    SET QuantidadeStock = QuantidadeStock - @QuantidadeVendida

    WHERE ProdutoID = @ProdutoID;


    -- Se todos os passos acima ocorrerem sem falhas, confirma as alterações
    definitivamente

    COMMIT TRANSACTION;

```

```

PRINT 'Transação Concluída com Sucesso! (COMMIT)';

END TRY

BEGIN CATCH

    -- Se ocorrer QUALQUER falha mecânica, lógica ou de rede no bloco TRY, desfaz
    tudo

    IF @@TRANCOUNT > 0

    BEGIN

        ROLLBACK TRANSACTION;

    END

    DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();

    RAISERROR(@ErrorMessage, 16, 1);

END CATCH

END;

GO

```

5. Simulação e Análise de Cenários Práticos

Para comprovar a eficiência do SGBD e das propriedades ACID, foram executados testes laboratoriais controlados.

5.1 Cenário 1: Carga Inicial de Dados

SQL

```

INSERT INTO Clientes (Nome, Telefone, Email, Endereco)

VALUES ('Marta João', '923000000', 'marta@gmail.com', 'Uíge');

```

```

INSERT INTO Produtos (NomeProduto, Categoria, PrecoUnitario, QuantidadeStock)

VALUES ('Sumo Compal', 'Bebidas', 1200.00, 10);

```

Análise: O produto "Sumo Compal" é inserido com o stock inicial de **10 unidades**.

Resultados			Mensagens		
NomeProduto	QuantidadeStock	PrecoUnitario			
1	Sumo Compal	10	1200.00		

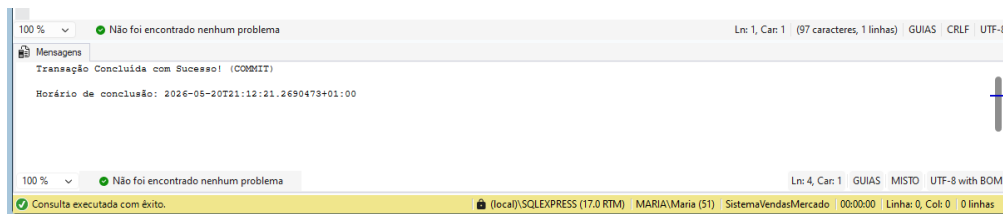
Consulta executada com êxito.

(local)\SQLEXPRESS (17.0 RTM) MARIA\Maria (70) SistemaVendasMercado 00:00:00 Linha: 1, Col: 1 1 linhas

5.2 Cenário 2: Execução com Sucesso (Fluxo do COMMIT)

SQL

```
EXEC sp_RegistarVenda @ClienteID = 1, @FuncionarioID = 1, @ProdutoID = 1,
@QuantidadeVendida = 3;
```



Análise: O cliente compra 3 unidades. O SGBD executa todas as instruções com sucesso. Ao consultar as tabelas, verifica-se que a venda foi registada, os detalhes foram criados corretamente e o stock do produto reduziu com precisão para 7 unidades. Isto demonstra a Atomicidade e a Durabilidade do processo através da instrução COMMIT.

(Inserir aqui o Print Screen da execução com sucesso e o resultado das tabelas)

5.3 Cenário 3: Aborto por Falha de Regra de Negócio (Fluxo do ROLLBACK)

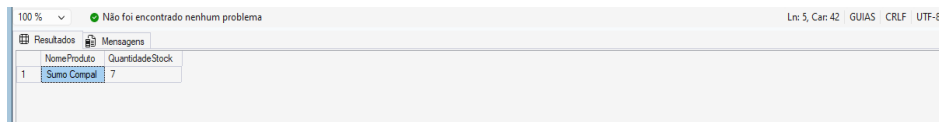
SQL

```
EXEC sp_RegistarVenda @ClienteID = 1, @FuncionarioID = 1, @ProdutoID = 1,
@QuantidadeVendida = 15;
```

-- Verificar que o stock não se alterou e continua seguro

```
SELECT NomeProduto, QuantidadeStock FROM Produtos WHERE ProdutoID = 1;
```

Análise: É efetuada uma tentativa de vender 15 unidades do mesmo produto, excedendo a quantidade disponível (7). A Stored Procedure deteta a violação da regra de integridade antes do processamento, bloqueia a inserção e dispara uma exceção. O estado do banco de dados permanece seguro e intacto em **7 unidades**, provando a eficácia da instrução ROLLBACK na preservação da **Consistência**.



	NomeProduto	QuantidadeStock
1	Sumo Compa	7

Figura 1 Resultado

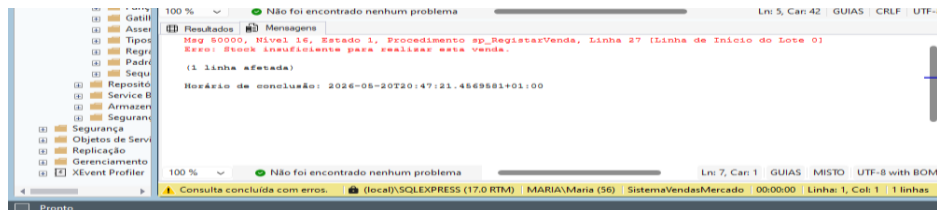


Figura 2 Mensagem erro vermelho ("Stock insuficiente")

6. Conclusão

A elaboração deste trabalho permitiu solidificar de forma prática os conceitos teóricos lecionados na cadeira de Sistemas de Gerenciamento de Banco de Dados. A implementação de transações controladas no SQL Server Management Studio provou ser indispensável para ambientes empresariais críticos, como o fluxo de vendas de um mercado.

Ficou evidente que, sem a correta manipulação dos comandos BEGIN TRANSACTION, COMMIT e ROLLBACK, o banco de dados ficaria vulnerável a estados inconsistentes, dados órfãos e falhas de concorrência. A utilização de Stored Procedures centraliza esta lógica diretamente no SGBD, garantindo alta performance, segurança de acessos e total conformidade com os princípios ACID.

